Diagrammi UML

- **Diagrammi di modellazione strutturale**
  - Diagrammi strutturali statici
    - Diagramma delle classi
  - Diagrammi di implementazione
    - Diagramma dei componenti
    - Diagramma di deployment (o dispiegamento)
    - Diagramma dei package
- **Diagrammi di modellazione dinamica**
  - Diagrammi di interazione
    - Diagramma di sequenza
      - Consigli di Scott Ambler
    - Diagramma di collaborazione (o comunicazione)

Presented with xmind

Diagrammi di implementazione

- Sono uno strumento utile per rappresentare gli aspetti dell'implementazione di un modello, come la struttura dei sottosistemi a vari livelli e la struttura fisica del sistema.
- Permettono di avere una visione d'insieme di come il sistema è stato progettato e come i vari componenti interagiscono tra loro.
- Ci sono tre tipi principali di diagrammi d'implementazione:
  - Diagramma dei componenti
  - Diagramma di deployment (o dispiegamento)
  - Diagramma dei package

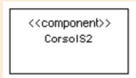
Presented with xmind

Diagramma dei componenti

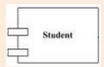
Definizione

- Utilizzato per rappresentare le unità modulari autonome del sistema, chiamate " componenti"
  - Un componente è un'unità modulare che ha una specifica interfaccia ben definita e che può essere sostituita con un'altra unità di uguale interfaccia
    - Questo rende i componenti sostituibili, modulari e riutilizzabili
  - Ogni componente è un'unità autonoma in un sistema, può fornire una o più interfacce, attraverso le quali è possibile accedere alle funzionalità che esso offre.
  - I componenti sono incapsulati.
    - Ovvero il suo interno è nascosto e inaccessibile a chi lo usa.
  - Le dipendenze sono progettate in modo che possa essere trattato nel modo più indipendente possibile, per migliorare la manutenibilità e la scalabilità del sistema
  - Gli elementi di un componente includono le interfacce, le dipendenze di uso, le porte e i connettori
    - Un'interfaccia è una dichiarazione di un insieme di operazioni che il componente deve fornire e definisce il comportamento del componente.
      - Possono essere fornite o richieste e definiscono l'insieme di una o più operazioni che il componente deve fornire o si aspetta di ricevere dal mondo esterno.
      - L'implementazione delle operazioni è fornita da una classe o da un componente, nei sistemi più complessi l'implementazione è fornita da un gruppo di classi e non da una singola classe.
    - Una dipendenza di uso è una relazione che indica che un elemento richiede un altro elemento per la sua implementazione
    - Una porta rappresenta il punto di interazione tra un componente e l'ambiente circostante
    - Un connettore connette due componenti o connette l'interfaccia esterna di un componente alla sua struttura interna

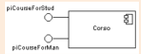
Notazione

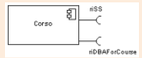
- I componenti sono rappresentati con
  - 

    Rettangoli con lo stereotipo <<component>>
  -

    L'icona "component in alto a destra"
  - 

    In UML 1.4 il simbolo per i component è questo
- Le interfacce possono essere rappresentate
  -

    Un rettangolo con lo stereotipo <<interface>> sopra il nome e con un comparto inferiore contenente la lista di operazioni
  - 

    Le interfacce fornite specificano i servizi che il componente offre al mondo esterno ed è modellata con un lollipop con il nome del componente vicino
  - 

    L'interfaccia richiesta specifica i servizi che il componente si aspetta dal mondo esterno ed è modellata con una "presa" con il nome del componente vicino.
  -

    Se due componenti o classi forniscono e richiedono la stessa interfaccia, le due notazioni si combinano.
    - La notazione pallino-presa suggerisce che l'interfaccia serve a mediare l'interazione tra i due componenti.

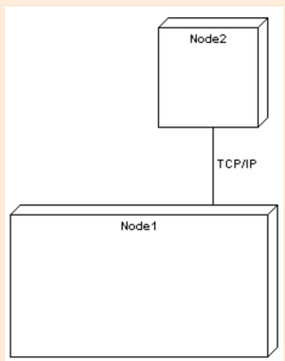
Presented with xmind

Diagramma di deployment (o dispiegamento)

Definizione

- Rappresentano le relazioni fisiche tra l' hardware e il software all'interno di un sistema
  - In particolare mostrano i nodi dove risiedono i componenti software durante il funzionamento del sistema e sono strettamente collegati ai diagrammi dei componenti
  - Gli elementi hardware che possono essere rappresentati nei diagrammi di dispiegamento includono
    - Computer (client, server)
    - Processori embedded
    - Dispositivi come sensori e periferiche
- È costituito da nodi e connessioni
  - Ogni nodo rappresenta un pezzo di hardware del sistema, ma può anche rappresentare una macchina virtuale
    - Spesso contengono uno o più componenti software
  - Le connessioni rappresentano il cammino di comunicazione usato dall'hardware per comunicare, indicando di solito il protocollo utilizzato (come TCP/IP)

Notazione



Presented with xmind

Diagramma dei package

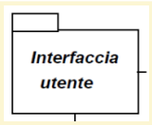
I pacchetti o package

- Sono sottosistemi all'interno del modello UML 1.0
  - Contengono
    - Altri pacchetti
    - Classi
    - Associazioni
    - Gerarchie di ereditarietà
- Rappresentano il modello del sistema a vari livelli di scala, dalle viste più grandi a quelle più dettagliate
- Servono principalmente per organizzare il modello in modo più chiaro e non hanno altri significati specifici, sebben possano essere definito dalla semantica del modello
  - Sono molto utili per suddividere il lavoro relativo a grandi progetti, basandosi su uno studio OOA/OOD
    - Object Oriented Analysis (OOA)
    - Object Oriented Design (OOD)
    - Il numero di classi in un modello OOD è compreso tra 30 e 50
      - 100 classi sono considerate molte
      - In caso di domini di problemi molto complessi, possono essere presenti quattro o cinque sottodomini, ognuno contenente 50-100 classi.
- Sono spesso derivati dal dominio del problema e possono essere suddivisi in modo naturale in insiemi di classi statiche che descrivono il dominio e dinamiche che descrive i dati che il sistema manipola in modo più dinamico.
  - Le gerarchie di ereditarietà semplice vanno di solito all'interno di un singolo pacchetto, così come le aggregazioni e le composizioni.
- Il principio fondamentale per la suddivisione dei pacchetti è quello di minimizzare le interdipendenze tra le classi appartenenti a pacchetti diversi, in modo da rendere il modello pi` u modulare ed efficiente.
  - Ogni pacchetto ha associato un diagramma delle classi che può contenere altri pacchetti, oltre alle classi e alle loro relazioni

Definizione

- Mostra solo i pacchetti e le loro relazioni.
- È stato introdotto esplicitamente in UML 2.0
- I pacchetti possono contenere diverse cose, tra cui classi, Casi d'Uso, componenti e altri elementi.
- Le relazioni tra pacchetti possono essere di diversi tipi, come l'uso comune, l'importazione, l'accesso, il tracciamento e la fusione
  - Per rappresentare queste relazioni tra i pacchetti ci sono anche vari stereotipi di uso comune
    - <<use>>
      - Semplice dipendenza
      - Si può anche omettere
    - <<import>>
      - Elementi pubblici di Supplier sono aggiunti come pubblici in Client
    - <<access>>
      - Elementi pubblici di Supplier sono aggiunti come privati in Client
    - <<trace>>
      - Rappresenta lo sviluppo di un package in un altro, più dettagliato
      - È una relazione tra modelli e non tra elementi
    - <<merge>>
      - Elementi pubblici di Supplier sono uniti a quelli di Client
      - Usata nel meta-modello e non in diagrammi ordinari
  - In questo contesto il Supplier espone elementi che possono essere utilizzati da altri pacchetti (Client)

Notazione



- I pacchetti sono rappresentati da un rettangolo contenente il nome del pacchetto e un rettangolo più piccolo che "spunta" in alto a sinistra, che indica le dipendenze tra i pacchetti.
- Dentro un pacchetto ci può essere il diagramma delle classi corrispondente

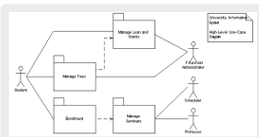


Diagramma dei package con casi d'uso

Presented with xmind

Diagrammi di modellazione dinamica

Modello dinamico

- Rappresenta il funzionamento del sistema durante l'esecuzione
- Definisce i possibili scenari di utilizzo del sistema in risposta a eventi esterni o a fasi significative
- Un esempio di strumento di descrizione dinamica sono i casi d'uso o le user stories
- I diagrammi che rappresentano l'invocazione di metodi possono essere utili per indicare la sequenza e la temporizzazione dell'invio dei messaggi.

L'analisi del funzionamento del sistema aiuta a verificare la completezza delle rappresentazioni delle classi in termini di attributi, relazioni e operazioni e la corretta assegnazione delle operazioni alle varie classi.

Non è necessario definire il modello dinamico nei minimi dettagli, in quanto i diagrammi dinamici possono essere utilizzati per descrivere il funzionamento del sistema nei casi più critici

La definizione dettagliata di tutte le operazioni del sistema può essere fatta costruendone il prototipo funzionante

Connessione tra oggetti

- Modella le dipendenze di un oggetto per quanto riguarda le elaborazioni, indicando la necessità di richiedere servizi per soddisfare le sue responsabilità.
- Mostra graficamente l'accesso di un oggetto da parte di un altro e la richiesta di un' operazione al primo
- È in stretto rapporto con le collaborazioni di un oggetto
- Questa connessione riguarda diagrammi UML in cui sono esplicitamente rappresentati degli oggetti e non le loro classi.

Diagrammi di interazione

- Diagramma di sequenza
  - Evidenziano i messaggi scambiati tra gli oggetti e ne danno la sequenza e la durata temporale.
- Diagramma di collaborazione (o comunicazione)
  - Mostrano e qualificano le connessioni fra oggetti e i messaggi scambiati tra questi

possono essere convertiti l'uno nell'altro entro certi limiti.

Presented with xmind

Diagramma di sequenza

Definizione

- Illustrano come gli oggetti interagiscono tra loro, aiutano a identificare responsabilità e operazioni dei singoli oggetti, e ad assegnarle correttamente alle classi.
- Include un certo numero di partecipanti e messaggi scambiati tra essi durante l'esecuzione dello scenario.
- Inizia con un partecipante (esterno) che invia un messaggi a un altro partecipante, iniziando un'elaborazione
- I messaggi vengono mostrati in ordine di tempo
- Può modellare flussi sequenziali, decisioni, cicli e parallelismi
  - Nonostante sia possibile modellare cicli e condizioni i diagrammi di sequenza non sono adeguati a farlo (meglio usare diagrammi di attività)

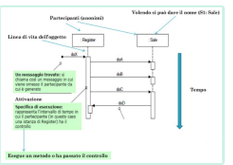
Partecipanti

- Rappresentati attraverso box o icone contenenti il loro nome e la loro classe
  - Sono rappresentati così: [nomeOggetto][:nomeClasse]
- Si deve inserire il nome degli oggetti quando vengono nominati in un messaggio o quando ci sono più oggetti appartenenti alla stessa classe.
- Da ogni partecipante si estende una linea tratteggiata chiamata lifeline
- Se un partecipante non è un oggetto (persona o sistema esterno) la sua esistenza è sempre garantita

Messaggi

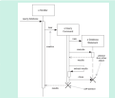
- L'interazione tra due oggetti è mostrata come un messaggio inviato da un oggetto all' altro
- Presuppone un link tra due oggetti
- È inviato eseguendo un metodo del mittente
- Deve corrispondere a un metodo definito nella classe del ricevente
- Sono rappresentati con una freccia tra la lifeline del mittente e del ricevente
- Devono avere almeno un nome e possono contenere anche parametri e condizioni
- Messaggio di ritorno
  - Rappresenta i valori resi dall'esecuzione di un messaggio
  - È in genere opzionale
    - Non usarlo se il valore reso è ovvio
    - Definirlo solo quando serve riutilizzarlo altrove
    - Piuttosto che usare un messaggio di ritorno è meglio definire i valori resi nel messaggio stesso
- Possono creare o distruggere un oggetto
  - In tal caso s possono usare gli stereotipi <<create>> e <<destroy>>

Notazione



- Messaggi sincroni
- Messaggi asincroni
- Messaggi di ritorno
- La creazione di un partecipante è rappresentata attraverso un messaggio diretto al box del partecipante stesso

- La distruzione di un partecipante è rappresentata da una grossa X all'interno della lifeline del partecipante



- Messaggi a se stesso

Cicli e condizioni

- Con i frame è possibile rappresentarli
- La notazione prevede l'utilizzo di rettangoli che racchiudono una regione del diagramma, detti frame di interazione
- Ogni frame ha un operatore.
  - (loop)[condizione o elementi del ciclo]
    Usato per rappresentare i loop per i cicli
  - (opt)[condizione]
    Usato per rappresentare una esecuzione opzionale di un if senza else
  - (alt)[condizione]
    Usato per rappresentare if/then/else
  - ref
    Usato per riferirsi a un altro diagramma

Presented with xmind

Consigli di Scott Ambler

Scott Ambler ha suggerito alcuni consigli pratici per lavorare con i diagrammi di sequenza:

- Chi sa usare i diagrammi di sequenza è bravo a pensare in modo logico e/o a scrivere software.
- Il diagramma di sequenza è una codifica visuale, anche quando si modella uno scenario d'uso a livello di sistema.
- È consigliabile iniziare identificando lo scopo del diagramma di sequenza.
- Si dovrebbe poi procedere a modellare il diagramma in piccoli incrementi, affrontando semplici scenari d'uso a livello di sistema o un singolo metodo/servizio a livello di un oggetto.
- Un diagramma troppo complesso non è utile, secondo l'esperienza di Scott Ambler.
- Per elaborare la logica del diagramma di sequenza, è consigliabile lavorare con almeno un'altra persona e disporre i partecipanti in alto man mano che li si trova.
- Si dovrebbero aggiungere automaticamente le lifeline degli oggetti, ma in genere non investire tempo ad aggiungere barre di attivazione.
- Il cuore del diagramma di sequenza è nei messaggi, che dovrebbero essere aggiunti uno alla volta mentre si elabora la logica dell'interazione.
- Raramente si dovrebbero indicare i valori di ritorno; invece, ai messaggi dovrebbero essere dati nomi esplicativi che chiariscono ciò che viene restituito.
- Mentre disegni il diagramma di sequenza, potresti identificare nuove responsabilità per le classi e gli oggetti, e talvolta anche nuove classi.
- Gli analisti agili seguono la pratica di creare diversi modelli contemporaneamente, cosa che può essere facilitata dagli strumenti CASE.
- Preferisco disegnare i messaggi che vanno da sinistra a destra e restituiscono i valori da destra a sinistra, anche se non sempre funziona bene con oggetti/classi complessi.
- Nella disposizione dei diagrammi di sequenza, disporre da sinistra a destra: attori, classi "controller", classi dell'interfaccia utente, classi di "business", classi di sistema e di persistenza, spesso semplifica la lettura e facilita la ricerca di problemi di modularità, come ad esempio classi dell'interfaccia utente che accedono direttamente alle classi di persistenza.
- Per mantenere l'agilità, disegna i diagrammi di sequenza su una lavagna per verificare la logica di un caso d'uso o per progettare un metodo o un servizio.
- Un errore comune è cercare di creare un insieme completo di diagrammi di sequenza per il proprio sistema. Ho visto team di progetto perdere mesi a creare diversi diagrammi di sequenza per ciascuno dei loro casi d'uso, uno per lo scenario principale e uno per ciascuno scenario alternativo.
- Crea un diagramma di sequenza solo quando hai una logica complessa che vuoi esaminare in dettaglio. Se la logica è semplice, il diagramma di sequenza non aggiungerà alcun valore: vai direttamente al codice.

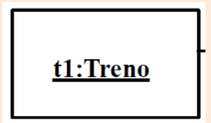
Presented with xmind

Diagramma di collaborazione (o comunicazione)

Definizione

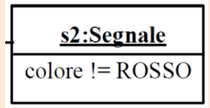
Mostrano e qualificano le connessioni fra oggetti e i messaggi scambiati tra questi.

Notazione



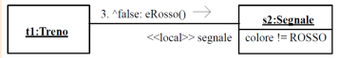
Gli oggetti sono rappresentati come rettangoli con all'interno il nome dell'oggetto e il nome della classe sottolineati per distinguersi dal simbolo di classe

Nella sezione superiore del rettangolo possono essere presenti ornamenti come "new", che indica che l'oggetto è stato appena creato



Nella sezione inferiore possono essere presenti vincoli o informazioni di stato.

Le connessioni tra oggetti sono rappresentate come linee continue tracciate tra gli oggetti



Alle estremità delle connessioni fra oggetti è possibile indicare in che modo l'oggetto è accessibile all'altro oggetto connesso

I possibili modi di accesso di un oggetto (ricevente) da parte di un altro (mittente) sono

- <<association>> — Il ricevente visibile in quanto associato al mittente (si implementa in genere con una variabile di istanza)
- <<global>> — Il ricevente è una variabile globale
- <<local>> — Il ricevente è una variabile locale dell' operazione eseguita dal mittente
- <<parameter>> — Il ricevente è un parametro dell'operazione eseguita dal mittente
- <<self>> — Il ricevente è lo stesso mittente

Gli invii di messaggi tra gli oggetti sono rappresentati da frecce parallele alle connessioni e i messaggi sono numerati e possono essere preceduti da condizioni, racchiuse tra parentesi quadre.

I messaggi possono rappresentare flussi multipli.

Presented with xmind